

# 第二次培训：Python 入门

## 目标

掌握 Python 基础语法：

- 变量与运算符
- 控制流
- 基本数据结构：数字、列表、元组、字典、集合
- 基本输入与输出
- 函数

## 为什么学 Python ？

- **门槛低、语法简洁** — 初学者上手快，能用更少代码表达想法。
- **用途广泛** — 自动化、数据分析、Web、脚本、教学、原型开发、游戏等。
- **生态丰富** — 成熟库（pandas、numpy、requests、pygame 等），能快速做出有价值的项目。
- **学习回报高** — 小项目、工具能显著提升学习成就感，带动进一步学习动力。
- **与其他语言互补** — Python 适合快速开发和原型设计，C++ 适合性能关键任务。两者结合能发挥各自优势。

## 怎么学 Python ？

**(1) 找一套合适的教程，跟着示例敲一遍：**

- 跟着示例代码“跑起来”，不要一开始就纠结细节。动手让程序能跑，才能有直观感受。
- 行动建议：选一个教程，把每个示例敲一遍并改一个小细节看结果。

**(2) 把语言当作工具 / 黑箱 —— 先学“用”：**

- 关注**输入** → **处理** → **输出**的流程。先学会用库和语法解决问题，再逐步研究底层实现。

### (3) 学会 Debug (调试):

- 逐步执行代码、观察变量值、用 print() 或 IDE 的断点调试。出问题就调试并查资料解决。

### (4) 能看懂并复用别人的代码:

- 先借鉴、改造已有实现，不必执着于“全部从零写起”。阅读好的开源代码能学习到实用模式。

### (5) 第一遍只求能用，后续遇到问题再深入:

- 第一轮以“能跑”为目标；当遇到性能、设计或理解瓶颈时，再深入研究原理和优化。

## 运行第一个 Python 程序

在命令行或 IDE 中输入以下代码并运行：

```
# Hello World
print("Hello, World!")
```

- # 后的内容是注释，解释代码作用，不会被执行。
- print() 是一个函数，作用是把括号内的内容输出到屏幕上。
- "Hello, World!" 是一个字符串，表示一段文本内容。
- 运行后，屏幕上会显示 Hello, World! 。

## 一个更复杂的示例：简易的学生成绩管理系统

实现要求：

### (1) 输入与输出

- 能够从用户输入学生姓名和多门成绩
- 输出学生成绩列表、平均分及其它统计信息

### (2) 变量与运算符

- 使用变量存储及格线、奖励分和成绩数据
- 使用算术运算计算总分与平均分

### (3) 控制流

- 根据平均分判断是否及格 (if/else)

### (4) 基本数据结构

- **数字**：存储成绩和平均分
- **列表**：存储多门成绩
- **元组**：存储固定信息（姓名、科目数）
- **字典**：存储完整学生档案（姓名、成绩列表、平均分）
- **集合**：去除重复成绩

### (5) 函数

- 定义函数 `add_bonus` 给所有成绩加奖励分，并返回新的成绩列表

### 教学目标：

- 演示 Python 基础语法、数据类型、输入输出、函数和控制流的综合应用
- 适合作为入门课程的课堂演示程序

# 学生成绩管理系统示例

# 1. 变量与运算符

```
passing_score = 60
```

```
bonus = 5
```

```
print("及格线:", passing_score, "奖励分:", bonus)
```

# 2. 输入与输出

```
name = input("请输入学生姓名: ")
```

```
scores_input = input("请输入该学生的成绩（用空格分隔）: ")
```

# 把输入字符串转为列表（数字），不用列表推导式

```
scores = []
```

```
for x in scores_input.split():
```

```
    scores.append(int(x))
```

# 3. 基本数据结构

# 数字：计算总分、平均分

```
total = 0
```

```
for s in scores:
```

```
    total = total + s
```

```
average = total / len(scores)
```

# 列表：存储多门成绩

```
print("成绩列表:", scores)
```

# 元组：存储固定信息（姓名，科目数）

```
student_info = (name, len(scores))
```

```
print("学生信息:", student_info)
```

```
# 字典：存储学生的完整档案
student_record = {
    "name": name,
    "scores": scores,
    "average": average
}
print("学生档案:", student_record)

# 集合：去除重复成绩
unique_scores = set(scores)
print("去重后的成绩:", unique_scores)

# 4. 控制流
if average >= passing_score:
    print(name, "平均分及格 🎉")
else:
    print(name, "平均分不及格 😬")

# 5. 函数
def add_bonus(scores, bonus):
    """给所有成绩加上奖励分"""
    new_list = []
    for s in scores:
        new_list.append(s + bonus)
    return new_list

# 调用函数
new_scores = add_bonus(scores, bonus)
print("加分后的成绩:", new_scores)
```

## 运行结果

```
PS Dir/>python student_scores.py
及格线：60 奖励分：5
请输入学生姓名：Alice
请输入该学生的成绩（用空格分隔）：80 90 80 70 90
```

得到如下输出：

成绩列表: [80, 90, 80, 70, 90]

学生信息: ('Alice', 5)

学生档案: {'name': 'Alice', 'scores': [80, 90, 80, 70, 90], 'average': 82.0}

去重后的成绩: {80, 90, 70}

Alice 平均分及格 🎉

加分后的成绩: [85, 95, 85, 75, 95]

## 1 变量

# 变量示例

```
passing_score = 60
```

```
bonus = 5
```

```
name = input("请输入学生姓名: ")
```

```
print("学生姓名:", name)
```

```
print("及格线:", passing_score, "奖励分:", bonus)
```

- 变量用来存储数据
- 类型示例: 数字、字符串
- 命名规则: 字母、数字、下划线, 不以数字开头

## 2 运算符

# 运算符示例

```
score1 = 80
```

```
score2 = 70
```

```
total = score1 + score2      # 算术运算
```

```
average = total / 2
```

```
print("总分:", total, "平均分:", average)
```

# 比较运算

```
print("平均分是否及格?", average >= passing_score)
```

- 算术运算: + - \* // % \*\*
- 比较运算: > < >= <= == !=

- 运算结果可用于条件判断

### 3 分支 (if/else)

```
if average >= passing_score:
    print(name, "平均分及格 🎉")
else:
    print(name, "平均分不及格 😞")
```

- if/else 判断条件执行不同语句
- 支持 elif 多分支
- 条件表达式返回布尔值

### 4 循环

```
scores_input = input("请输入该学生成绩（用空格分隔）：")
scores = []
for x in scores_input.split():
    scores.append(int(x))
```

- for 遍历列表
- while 循环可实现带条件的重复执行
- break / continue 可控制循环流程，如提前退出或跳过

一个 while 循环和 break/continue 示例：

```
# while 示例
count = 0
while count <= 5:
    print("循环次数:", count)
    count = count + 1

    if count == 2:
        print("跳过本次循环")
        continue

    if count == 3:
        print("退出循环")
        break

    print("当前 count 值:", count)
```

运行结果：

```
PS Dir/>python while_demo.py
```

循环次数：0

当前 count 值：1

循环次数：1

跳过本次循环

循环次数：2

退出循环

- 第 0 次循环正常执行
- 第 1 次循环触发 continue，跳过打印当前 count
- 第 2 次循环触发 break，循环提前结束

## 5 数字

```
average = total / len(scores)
print("平均分:", average)
```

- 数字类型：int、float、double、bool，例如：
  - int： 80, -5, 0
  - float 或 double： 3.14, -0.001
  - bool： True, False
- 可用于算术运算
- 可作为控制流条件判断

## 6 列表

```
print("成绩列表:", scores)
>>> 成绩列表: [80, 90, 80, 70, 90]
```

- 列表的标准形式： [元素1, 元素2, 元素3]
- 列表是可变序列
- 可存储多门成绩
- 支持**索引** scores[i]、**切片** scores[start:end]、**增删改查**等操作

## 7 集合

```
unique_scores = set(scores)
print("去重后的成绩:", unique_scores)
>>> 去重后的成绩: {80, 90, 70}
```

- 基本形式： {元素1, 元素2, 元素3}
- 集合自动去重
- 支持交集、并集、差集等集合运算
- 元素无序、不重复

## 8 元组

```
student_info = (name, len(scores))
print("学生信息:", student_info)
>>> 学生信息: ('Alice', 5)
```

- 基本形式：(元素1, 元素2, ..., 元素3)
- 元组不可变
- 用于存储固定信息
- 可以作为字典的键

## 9 字典

```
student_record = {
    "name": name,
    "scores": scores,
    "average": average
}
print("学生档案:", student_record)
>>> 学生档案: {'name': 'Alice', 'scores': [80, 90, 80, 70, 90], 'average': 82.0}
```

- 字典存储**键值对**
- 支持 keys(), values(), items()
- 遍历字典可访问键和值

## 10 输入与输出

```
name = input("请输入学生姓名: ")
print("欢迎,", name)
>>> 请输入学生姓名: Alice
>>> 欢迎, Alice
```

- input() 获取用户输入
- print() 输出信息

- 支持字符串拼接与格式化 (f-string)

## 1 1 函数

```
def add_bonus(scores, bonus):  
    new_scores = []  
    for s in scores:  
        new_scores.append(s + bonus)  
    return new_scores
```

```
new_scores = add_bonus(scores, bonus)  
print("加分后的成绩:", new_scores)
```

- 使用 `def` 定义函数
- 函数封装可复用逻辑
- 支持参数和返回值

## 调试小技巧

- **逐步执行**：用 `print()` 输出变量值，或使用 IDE 断点调试，逐行执行代码
- **使用调试工具**：利用 IDE 提供的调试工具，单步执行代码，观察变量变化
- **查阅文档**：遇到不懂的函数或库，查阅官方文档
- **简化问题**：把复杂问题拆成小块，逐步调试
- **搜索解决方案**：利用搜索引擎查找类似问题的解决方法
- **保持耐心**：调试是编程的重要部分，遇到问题时保持冷静，逐步排查

## 入门阶段使用 AI 工具的注意事项

### (1) 关闭 AI 生成和改错功能

- 初学时不要依赖 AI 自动生成或修改代码

### (2) 遇到问题先使用传统调试方法

- **调试工具**：逐步执行程序，观察变量和逻辑
- **搜索资料**：将报错信息粘贴到搜索引擎，查找博客和教程
- **最后再求助 AI**：在以上方法都无法解决时，再使用 AI

这样可以充分暴露问题，锻炼解决问题的能力。通过这些过程，即使很多信息看似无关，也能帮助你积累语言语感和 debug 能力

### (3) 甄别 AI 的输出

- AI 可能会给出错误或不完整的解释
- 使用前务必核实内容

### (4) 实际开发中合理使用 AI

- 当具备基础能力后，可利用 AI 提升开发效率
- AI 作为辅助工具，而非完全依赖

养成良好习惯，独立思考，才能真正提升编程能力